

Enterprise Flutter CI/CD Implementation

1. Objective

Build a complete, production-ready, on-premises CI/CD platform for the Flutter application using:

- Flutter
- Jenkins
- Fastlane
- Docker
- GitLab as the initial source-control provider
- Android
- iOS
- Web
- Testing, Staging and Production environments
- Debug and Release build configurations
- Automated testing and quality gates
- Automated versioning
- Automated application signing
- Automated artifact generation
- Artifact repository/storage
- Cloud distribution
- Future integration with GuardSquare application protection
- On-premises Jenkins infrastructure
- Reproducible and version-controlled CI/CD infrastructure

The architecture must not be designed only for the current application. It must be reusable and extensible so that additional Flutter applications can use the same CI/CD platform later.

The final system should allow an authorized user to select an environment, platform and build mode and have Jenkins perform the complete process automatically.

The target platforms are:

Android

iOS

Web

The target environments are:

Testing

Staging

Production

The target build modes are:

Debug
Release

Therefore, the CI/CD architecture must support:

3 environments
2 build modes
3 platforms

2. Important Architectural Principle

Separate the CI/CD system into these layers:

- 1- Source Control
- 2- Jenkins Orchestration
- 3- Android/iOS/Web
- 4- Fastlane
- 5- Quality Gates
- 6- Build Artifacts
- 7- GuardSquare
- 8- Artifact Storage
- 9- Cloud Distribution (Firebase distribution/TestFlight/Google Play Beta)

* GuardSquare integration should initially be designed as a dedicated pipeline stage/interface even if the actual protection/shielding integration is implemented in a later phase.

Do not mix application compilation, protection, packaging and deployment into one large Jenkins script.

Each responsibility must have a clear boundary.

3. Platforms

The CI/CD system must support:

Android

Generate appropriate:

APK

AAB

depending on the destination.

Typical usage:

Testing Debug -> APK

Testing Release -> APK

Staging Debug -> APK

Staging Release -> AAB/APK

Production Debug -> APK/build validation

Production Release -> AAB

The exact mapping must be configurable.

iOS

Generate:

IPA

for deployable builds.

Support:

Debug

Release

for:

Testing

Staging

Production

Web

Generate the Flutter web application.

For example:

`flutter build web --release`

The resulting web distribution directory must be treated as a versioned CI artifact.

The Web pipeline must support:

Testing Debug

Testing Release

Staging Debug

Staging Release

Production Debug

Production Release

4. Web Deployment Architecture

Web must be treated separately from mobile distribution.

The architecture should support deployment to an appropriate web hosting platform.

The target deployment location must be configurable.

Possible destinations include:

Firebase Hosting

Nginx

Apache

On-premises web server

Cloud object storage + CDN

Other approved hosting platform

For the initial implementation, use the organization's selected hosting destination.

The pipeline should conceptually perform:

Flutter Build -> Web Artifact -> Artifact Validation -> Testing/Staging/Production Hosting

Do not directly deploy an unverified Jenkins workspace.

5. Environment Matrix

The implementation should use reusable pipeline logic rather than creating 18 independent Jenkins jobs.

6. Environment Configuration

Create three environments:

testing

staging

production

Environment configuration must be externalized from application business logic.

Use appropriate Flutter mechanisms such as:

--dart-define

Flutter flavors

Android product flavors

iOS schemes/configurations

xcconfig

environment-specific configuration

The final solution must clearly document which mechanism is used.

7. Centralized Flutter Configuration

Create a central configuration abstraction.

For example:

```
class AppConfig {  
  static const environment =  
    String.fromEnvironment('ENVIRONMENT');  
  
  static const apiBaseUrl =  
    String.fromEnvironment('API_BASE_URL');  
  
  static const appName =  
    String.fromEnvironment('APP_NAME');  
}
```

The actual implementation may differ.

Configuration should support values such as:

ENVIRONMENT

API_BASE_URL

APPLICATION_ID

APP_DISPLAY_NAME

FIREBASE_PROJECT

FEATURE_FLAGS

Secrets must never be passed as ordinary application configuration.

8. Environment-Specific Application Identity

Testing and Staging should preferably have different identifiers from Production.

For example:

Android:

com.company.app.testing
com.company.app.staging
com.company.app

iOS:

com.company.app.testing
com.company.app.staging
com.company.app

Web:

testing.example.com
staging.example.com
example.com

The actual values should come from project configuration.

This allows Testing and Staging applications to coexist on the same mobile device.

9. Android Configuration

Implement Android product flavors:

testing
staging
production

Each flavor must support:

debug
release

Configure environment-specific:

- Application ID
- App name
- App icon
- Firebase configuration
- API endpoint
- Deep links
- Push notification configuration
- Signing
- Other environment-specific values

Avoid duplicating entire Android projects for each environment.

10. iOS Configuration

Implement equivalent iOS configurations using:

- Schemes
- Build configurations
- .xcconfig
- Bundle identifiers

- Signing configuration

The naming must remain consistent with Android:

Testing

Staging

Production

The iOS scheme/configuration names must be documented.

11. Web Configuration

Web must use the same environment abstraction.

For example:

Testing:

API_BASE_URL=https://api-testing.example.com

Staging:

API_BASE_URL=https://api-staging.example.com

Production:

API_BASE_URL=https://api.example.com

Do not embed environment-specific URLs throughout the Dart code.

The Web build must use the same configuration source as the mobile applications wherever possible.

12. Debug vs Release

The pipeline must explicitly receive:

ENVIRONMENT

BUILD_MODE

PLATFORM

For example:

ENVIRONMENT=staging

BUILD_MODE=release

PLATFORM=android

Do not infer the environment from the build mode.

Do not infer production from the branch name inside application code.

13. Jenkins Architecture

Deploy Jenkins on-premises.

Use:

Jenkins Controller

1- Linux Docker Agent

2- macOS iOS Agent

3- Optional Web Agent

The Jenkins controller must not execute resource-intensive mobile builds.

14. Linux Docker Agent

The Linux Docker build agent should support:

- Flutter
- Dart
- Android SDK
- Android Build Tools
- JDK
- Gradle
- Ruby
- Bundler
- Fastlane
- Git
- Node.js if required
- Firebase CLI if required
- Security tooling
- GuardSquare tooling if applicable

The image must be version-controlled.

15. macOS iOS Agent

iOS builds require macOS and Apple's toolchain.

Use an on-premises Mac as a Jenkins agent.

It must contain:

- macOS
- Xcode
- Xcode Command Line Tools
- Flutter SDK
- Dart
- Ruby
- Bundler
- Fastlane
- CocoaPods
- Git

Required signing assets

The macOS machine must be dedicated or appropriately isolated for CI workloads.

Do not attempt to build iOS in the Linux Docker container.

16. Web Build Agent

Web can run on the Linux Docker agent unless there is a technical reason to separate it.

The Web build image should contain:

- Flutter
- Dart
- Git
- Ruby/Fastlane where required
- Node.js where required
- Firebase CLI where required

The Web pipeline should ideally reuse the same Flutter Docker image where practical.

17. Dockerization

The CI tooling must be containerized wherever technically possible.

Create version-controlled Dockerfiles.

For example:

Docker

- flutter-android
- Dockerfile
- flutter-web
- Dockerfile
- tools

iOS cannot be containerized using Linux Docker because Xcode requires macOS.

Document this explicitly.

18. CI Docker Image Versioning

Never use:

latest

as the only CI image reference.

Use immutable versions such as:

company/flutter-ci:1.0.0.0.0

company/flutter-ci:1.1.0.0.0

The application pipeline must explicitly select a known version.

Record the image version in the build manifest.

Use the versioning mechanisms shared earlier.

19. Dependency Reproducibility

Use locked dependencies.

Flutter:

pubspec.lock

Ruby:

Gemfile

Gemfile.lock

Android dependencies should use controlled versions.

The CI environment must not unexpectedly upgrade:

Flutter

Dart

JDK

Gradle

Android SDK

Fastlane

Ruby

Xcode

CocoaPods

without an intentional infrastructure change.

20. Fastlane Responsibilities

Fastlane should handle mobile-specific operations.

Jenkins should orchestrate the workflow.

Use:

Jenkins

- Fastlane
 - o Android build
 - o iOS build
 - o Signing
 - o Distribution

Do not place extensive Gradle/Xcode/deployment logic directly in Jenkinsfile.

21. Fastlane Lanes

Create reusable lanes such as:

test

analyze

build_android

build_ios

deploy_android_testing

deploy_android_staging

deploy_android_production

deploy_ios_testing

deploy_ios_staging

deploy_ios_production

Prefer generic parameterized lanes over duplicated lanes.

For example:

```
build_android(  
  environment,  
  build_mode  
)
```

rather than six separate implementations.

22. Fastlane Deployment Destinations

Fastlane should provide the mobile distribution abstraction.

The intended destinations are:

Testing

Firebase App Distribution

Staging

Potentially:

Firebase App Distribution

Google Play testing/beta track

TestFlight

depending on the organization's release process.

Production

Google Play production

Apple App Store

Production deployment must require approval.

23. Firebase App Distribution

Implement Firebase App Distribution for appropriate Testing/Staging builds.

The pipeline must be able to:

Build

Sign

Package

Upload

Assign testers/groups

Publish release notes

Return distribution URL

The release notes should contain useful information such as:

Version

Build number

Environment

Git commit

Jenkins build

Changes

Firebase credentials must be stored securely in Jenkins.

Do not commit Firebase service-account credentials to Git.

24. TestFlight

Implement TestFlight deployment for appropriate iOS Testing/Staging builds.

The pipeline must be able to:

Build IPA

Sign IPA

Upload to App Store Connect

Associate version/build

Provide release notes

Use appropriate App Store Connect API credentials.

Do not store private keys in Git.

The deployment should support separate environments/build configurations as required by the application's bundle identifiers.

25. Google Play Testing/Beta

Implement Google Play deployment for Staging or other approved release environments.

Use the appropriate Play Console testing track, for example:

Internal testing

Closed testing

Open testing

The selected track must be configurable.

The pipeline must:

Build AAB

Sign AAB

Upload

Select track

Publish release

Return release information

Google Play service credentials must be stored securely.

26. Production Mobile Distribution

Production Release should support:

Android -> Google Play Production

iOS -> App Store

However, the pipeline must have an approval gate before production deployment.

The workflow should be:

Build -> Sign -> Publish artifact -> Approve -> Deploy

27. Web Distribution

Web deployment should use the selected hosting platform.

If Firebase Hosting is selected, the flow may be:

Flutter Build Web -> Validate -> Artifact -> Hosting

Support separate targets:

Testing

Staging

Production

For example:

testing.example.com

staging.example.com

example.com

The exact hosting implementation must be configurable.

28. Cloud Credentials

Cloud credentials must be stored in Jenkins Credentials or an approved enterprise secret manager.

Examples:

Firebase credentials

Google Play service account

App Store Connect API key

Apple signing credentials

Artifact repository credentials

Never put these values in:

Jenkinsfile

Dockerfile

Fastfile

Git repository
application source

29. Future GuardSquare Integration

The CI/CD architecture must explicitly prepare for application protection using GuardSquare.

The protection process should be designed as a separate pipeline stage:

Build -> Test -> Sign -> GuardSquare Protection -> Protected Artifact Artifact Validation -> Distribution

Do not tightly couple the GuardSquare implementation into every build step.

30. GuardSquare Integration Boundary

Create a clear abstraction such as:

protect_android()

protect_ios()

or an equivalent script/service interface.

The pipeline should eventually be able to execute:

unprotected artifact -> GuardSquare -> protected artifact

The implementation must allow GuardSquare to be enabled/disabled by configuration.

For example:

PROTECTION_ENABLED=true

or:

PROTECTION_ENABLED=false

The exact implementation can differ.

31. GuardSquare Requirements

When GuardSquare integration is implemented, it must support appropriate protection for:

Android

iOS

The developer must work from the organization's licensed GuardSquare products and approved configuration.

Do not assume the same protection mechanism applies identically to Android and iOS.

The GuardSquare configuration must be version-controlled where safe.

Secrets and license credentials must be stored securely.

32. Artifact Naming

Artifacts must contain:

Application

Platform

Environment

Build Mode

Version

Build Number

Protection Status

Example:

company-app-android-staging-release-1.5.0.0.0+245-protected.aab

company-app-ios-staging-release-1.5.0.0.0+245-protected.ipa

company-app-web-staging-release-1.5.0.0.0+245.zip

The exact convention may differ.

33. Artifact Repository

Do not rely exclusively on Jenkins artifacts.

Use a proper artifact repository/storage solution such as:

Nexus

Artifactory

MinIO/S3-compatible storage

Other approved enterprise repository

Store:

APK

AAB

IPA

Web ZIP/archive

Build manifest

Checksums

Release notes

Optional symbol/debug files

34. Build Manifest

Every build must generate a machine-readable manifest.

Example:

```
{
  "application": "company-app",
  "platform": "android",
  "environment": "staging",
  "buildMode": "release",
  "version": "1.5.0",
  "buildNumber": "245",
  "gitCommit": "abcdef123",
  "gitBranch": "staging",
  "gitTag": "v1.5.0",
  "jenkinsJob": "company-app",
  "jenkinsBuild": "245",
  "flutterVersion": "...",
  "dartVersion": "...",
  "fastlaneVersion": "...",
  "cilimage": "...",
  "guardSquareEnabled": true,
  "buildTimestamp": "..."
}
```

}

The exact format can differ.

35. Checksums

Generate SHA-256 checksums for every final artifact.

Store:

artifact

artifact.sha256

build-manifest.json

This allows artifact integrity to be verified.

36. Debug Artifacts

Debug builds must still be treated as first-class CI outputs.

The pipeline must not assume only Release matters.

Debug builds should be useful for:

Developer testing

QA debugging

Internal validation

Environment validation

Where possible, retain debugging/symbol information appropriately.

37. Release Artifacts

Release artifacts must be suitable for their intended distribution destination.

The pipeline must verify:

Correct environment

Correct package/bundle identifier

Correct signing

Correct version

Correct build number

Correct platform

Correct build mode

before distribution.

38. Production Protection

Production must have additional safeguards.

A production build must validate:

Production environment

Production identifiers

Production API configuration

Release mode

Correct signing

Correct version

Correct tag/commit

No prohibited debug settings

Security checks passed
GuardSquare status

39. Source Control

GitLab is the initial SCM.

However, Jenkins must not be architecturally dependent on GitLab.

Keep GitLab-specific behavior limited to:

- Webhook
- Authentication
- Merge Request status
- GitLab UI integration

The actual pipeline should operate on generic Git information:

- commit
- branch
- tag
- repository

40. Future SCM Support

The design must allow:

- GitLab -> GitHub
- GitLab -> Bitbucket
- GitLab -> another Git server

without rewriting:

- Fastlane
- Flutter configuration
- Docker images
- Android build
- iOS build
- Web build
- GuardSquare integration
- Deployment logic

Provide documentation describing how another Git provider would be configured.

41. Deployment Documentation

Document exactly how to deploy:

- Testing Android
- Testing iOS
- Testing Web

- Staging Android
- Staging iOS
- Staging Web

- Production Android
- Production iOS

Production Web

For every destination, document:

Trigger

Build

Protection

Signing

Deployment

Validation

Rollback

42. Final Deliverables

The developer must deliver all of the following:

Source Control

Jenkinsfile

Fastfile

Gemfile

Gemfile.lock

Dockerfiles

Scripts

Pipeline configuration

Environment configuration

Jenkins

Jenkins configuration

Agents

Credentials definitions/documentation

Pipelines

Shared Library if applicable

Webhook configuration

Mobile

Android flavors

iOS schemes/configurations

Android signing

iOS signing

Fastlane automation

Web

Web environment configuration

Web build automation

Web artifact packaging

Web deployment automation

Distribution

Firebase App Distribution

TestFlight

Google Play testing/beta

Google Play production

App Store production

Web hosting

as applicable to the organization's approved targets.

Future Protection

GuardSquare pipeline stage/interface
GuardSquare documentation
Configuration strategy
Failure handling

Infrastructure

Docker
Jenkins
Linux agent
macOS agent
Artifact repository
Docker registry

Documentation

Architecture
Setup
Operations
Security
Signing
Releases
Deployment
GuardSquare
Troubleshooting
Rollback
Disaster recovery
SCM migration

43. Definition of Done

The implementation is complete only when:

- Jenkins is running on-premises.
- Jenkins Controller is separated from build agents.
- Android builds execute through Docker.
- Web builds execute through Docker.
- iOS builds execute on an on-premises macOS agent.
- Flutter version is controlled.
- Fastlane version is controlled.
- Ruby dependencies are locked.
- Android dependencies are controlled.
- Testing exists.
- Staging exists.
- Production exists.
- Debug exists for all platforms/environments.
- Release exists for all platforms/environments.
- Android flavors are implemented.
- iOS schemes/configurations are implemented.
- Web environment configuration is implemented.
- Environment configuration is centralized.
- GitLab integration works.

- SCM-specific logic is isolated.
- Merge Request CI works.
- Manual Jenkins builds work.
- Automatic branch/tag triggers work.
- Automated versioning works.
- Automated build numbering works.
- Android signing is automated.
- iOS signing is automated.
- Web artifacts are generated.
- Unit tests run automatically.
- Widget tests run automatically.
- Integration tests run where available.
- Static analysis runs automatically.
- Formatting checks run automatically.
- Security/secret scanning runs.
- Artifacts are stored outside Jenkins workspace.
- Build manifests are generated.
- SHA-256 checksums are generated.
- Firebase App Distribution integration works.
- TestFlight integration works.
- Google Play testing/beta integration works.
- Production Google Play deployment works.
- Production App Store deployment works.
- Web deployment works.
- Production approval is implemented.
- Production credentials are protected.
- GuardSquare integration point is implemented.
- GuardSquare failure stops deployment.
- Protected artifacts are validated before distribution.
- Previous artifacts are retained.
- Deployment failures can be retried without unnecessary rebuilding.
- Rollback/recovery procedures are documented.
- Jenkins is monitored.
- Certificates/credentials have a renewal process.
- CI infrastructure is documented.
- All 18 build combinations have been tested.
- Distribution paths have been demonstrated.
- Security tests have been completed.
- Source-control migration procedure has been documented.
- A new developer can operate the complete CI/CD system using the documentation.

44. Final Expected User Experience

The final Jenkins UI should provide an experience similar to:

FLUTTER MOBILE/WEB CI/CD

Environment:

Testing

Build Mode:

Release

Platform:

Android + iOS + Web

Protection:

Enabled

Deploy:

Yes

Version:

Automatic

Build Number:

Automatic